Infrastructure as Code Foundations

Part 2 — From IaC design to automated delivery

SEAtS | 424812

University of Southampton

Intended Learning Outcomes

By the end of this session you will be able to:

1. Decompose an IaC system into modules with clear, versioned interfaces.
2. Explain how IaC integrates provider APIs, third-party providers, and reusable internal IP.
3. Use formal specification (schemas, typed variables, policy-as-code) to make infrastructure enforceable.
4. Plan a testing strategy for IaC, from static analysis to security tests.
5. Design a CI/CD and GitOps pipeline for infrastructure changes.
6. Distinguish provisioning, deployment, orchestration and configuration management.
7. Identify common IaC risks and anti-patterns and propose mitigations.

© University of Southampton

Agenda

1. Designing IaC systems: modules, interfaces, providers, third-party IP.
2. Service delivery platform and formal specification.
3. Configuration examples in code.
4. Testing IaC systems.
5. Build systems, CI/CD, GitOps, and managing development.
6. Provisioning, deployment, orchestration, configuration management.
7. Risks, anti-patterns, and summary.

© University of Southampton

Designing IaC Systems

Decomposing into Blocks

Example module folders:

- `/networking`
- `/identity`
- `/compute`
- `/database`
- `/monitoring`

Identifying Interfaces

Interfaces via:

- Input variables.
- Output variables.
- Remote state references.

Interfaces must be clear, versioned, documented, and stable.

Designing APIs & Protocols

IaC interacts with:

- Cloud provider APIs.
- Terraform provider schemas.
- Kubernetes manifests/CRDs.

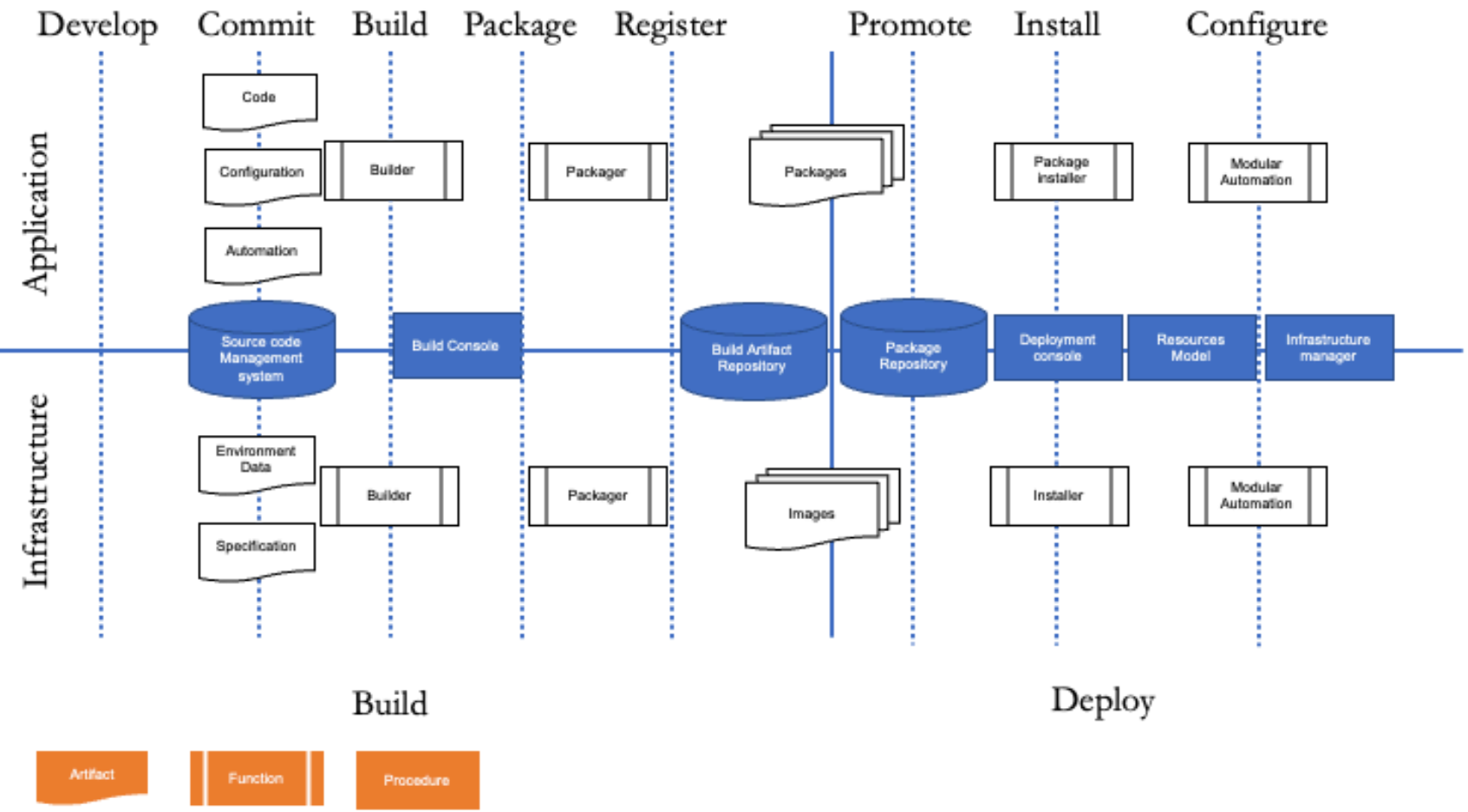
Integrating Providers & IP

- Cloud providers (AWS, Azure, GCP).
- Third-party providers (Cloudflare, Datadog).
- Internal libraries reused across teams.

Managing Third-Party IP

- Version pinning and lifecycle management.
- Licensing and compliance considerations.
- Security scanning of modules/providers.

© University of Southampton



Service Delivery Platform

- **Artifact:** Versioned, tested, deployed units.
- **Procedures (Imperative):** Commands to produce desired state (e.g., Bash).
- **Functions (Declarative):** Define desired state; tooling conforms system (e.g., SQL, makefiles).

© University of Southampton

Formal Specification in IaC

- JSON/YAML schemas (CloudFormation, Kubernetes).
- Terraform variable typing and module interfaces.
- Policy-as-Code (OPA/Rego) for compliance.
- Cloud policies (AWS Config, Azure Policy) to enforce encryption, logging, segmentation.

Formalisation supports reproducibility and enforcement.

© University of Southampton

YAML Examples

```
Vagrant.configure("2") do |config|
  config.vm.box = "peru/ubuntu-18.04-desktop-amd64"
  config.vm.box_version = "2.3"
  config.vm.provider "virtualbox" do |vb|
    vb.gui = true
    vb.memory = "1024"
    vb.name = "ubuntudesktopscyther"
  end
end
```

```
servers = YAML.load_file('servers.yaml')
Vagrant.configure(VAGRANTFILE_API_VERSION) do |config|
  servers.each do |server|
    config.vm.define server["name"] do |srv|
      srv.vm.box = server["box"]
      srv.vm.network "private_network", ip: server["ip"]
      srv.vm.provider :virtualbox do |vb|
        vb.name = server["name"]
        vb.memory = server["ram"]
      end
    end
  end
end
```


© University of Southampton

Testing IaC Systems

Testing types:

1. Static testing (linting: tflint, checkov; policy-as-code).
2. Unit tests (Terratest, Molecule).
3. Integration tests (ephemeral deployments).
4. Acceptance tests (VPC routing, IAM permissions).
5. Security tests (CSPM, IaC scanning).

© University of Southampton

Build Systems & CI

Standard Pipeline

1. Validate (`terraform validate`).
2. Lint (tflint, checkov).
3. Plan (static analysis, diffs).
4. Approvals (human-in-loop).
5. Apply (automated, controlled).
6. Post-deploy tests (Terratest, smoke tests).

Continuous Deployment (GitOps)

- ArgoCD / Flux for declarative sync.
- Atlantis, Spacelift, Terraform Cloud for collaboration.
- Automated rollbacks based on policy drivers.

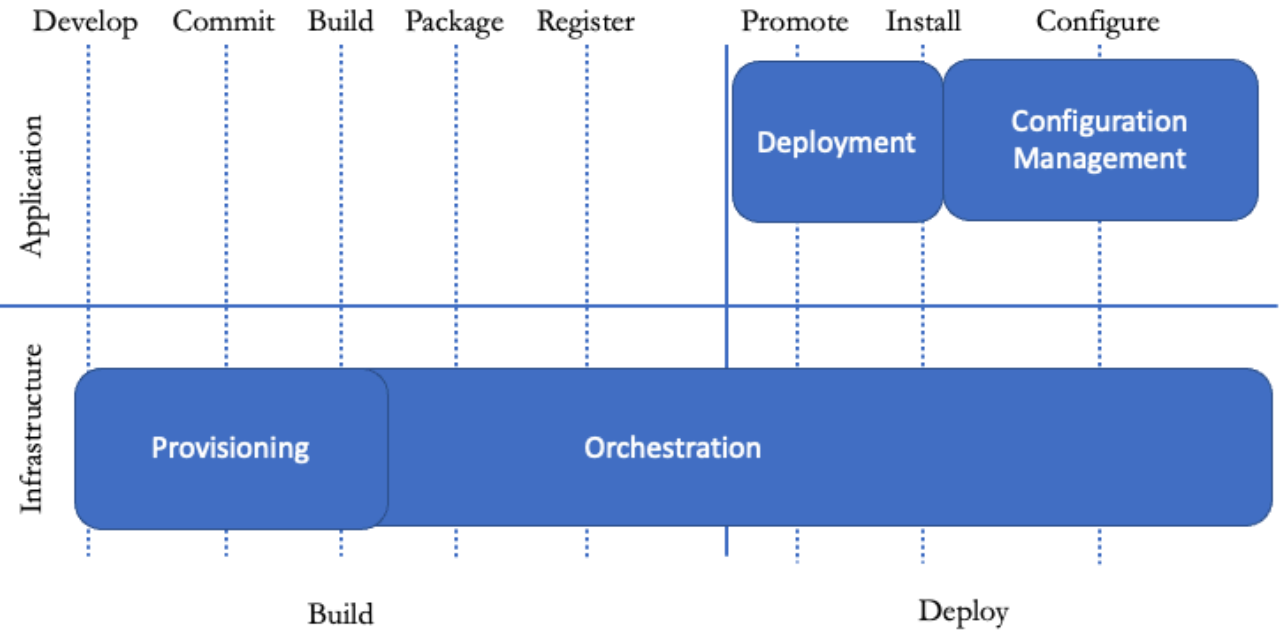
Managing Development

Methodologies: Agile, DevOps, GitOps, infrastructure release management.

- Multi-team decomposition, module backlogs, environment versioning.
- Branch protection, review bots (tfsec), semantic versioning.
- Automated changelogs and release notes.

© University of Southampton

Deployment / Orchestration / CM



- **Provisioning:** Prepare server (hardware, OS, services, network).
- **Deployment:** Automated app deployment & upgrades.
- **Orchestration:** Coordinated operations across systems.
- **Configuration Management:** Maintain app dependencies.

© University of Southampton

Risks & Anti-Patterns

- Configuration drift.
- Hard-coded secrets.
- Unversioned remote state.
- No testing or code reviews.
- Monolithic Terraform files, parallel changes without locking.
- Black-box modules with no documentation.

Mitigation: code reviews, state locking, policy-as-code, decomposition.

© University of Southampton

Summary

- Decompose infrastructure into modules whose interfaces (inputs, outputs, remote state) are clear, versioned and stable.
- Manage providers and third-party modules with version pinning, licence checks and security scanning.
- Formal specification (schemas, typed variables, policy-as-code) makes infrastructure reproducible and enforceable.
- Test IaC at every level: static, unit, integration, acceptance and security.
- CI/CD pipelines (validate → lint → plan → approve → apply → test) and GitOps keep changes controlled and auditable.
- Mitigate anti-patterns (drift, secrets, unlocked state, monoliths) through reviews, locking, policy-as-code and decomposition.

© University of Southampton